Rust製・省メモリブラウザ 設計書

Oct 26, 2026 · @YUTAKA

概要と設計目標

本設計（仮称 **Kumo**）は、レンダリングエンジンからUIまで全てRustで書き、1タブ表示時の常駐メモリ70MB以下を目指すブラウザです。Chromium系が1タブあたり100MB超を使う主因（サイトごとのプロセス、JIT、無制限キャッシュ）を、設計段階で一つずつ外します。

数値目標（RSS、1080p、x86-64 Linux）

シナリオ	目標
起動直後・空白タブ	30MB以下
Wikipedia記事1タブ	70MB以下
一般的なサイト10タブ（9タブはバックグラウンド）	250MB以下
凍結済みバックグラウンドタブ1枚あたり	5MB以下

主な対象

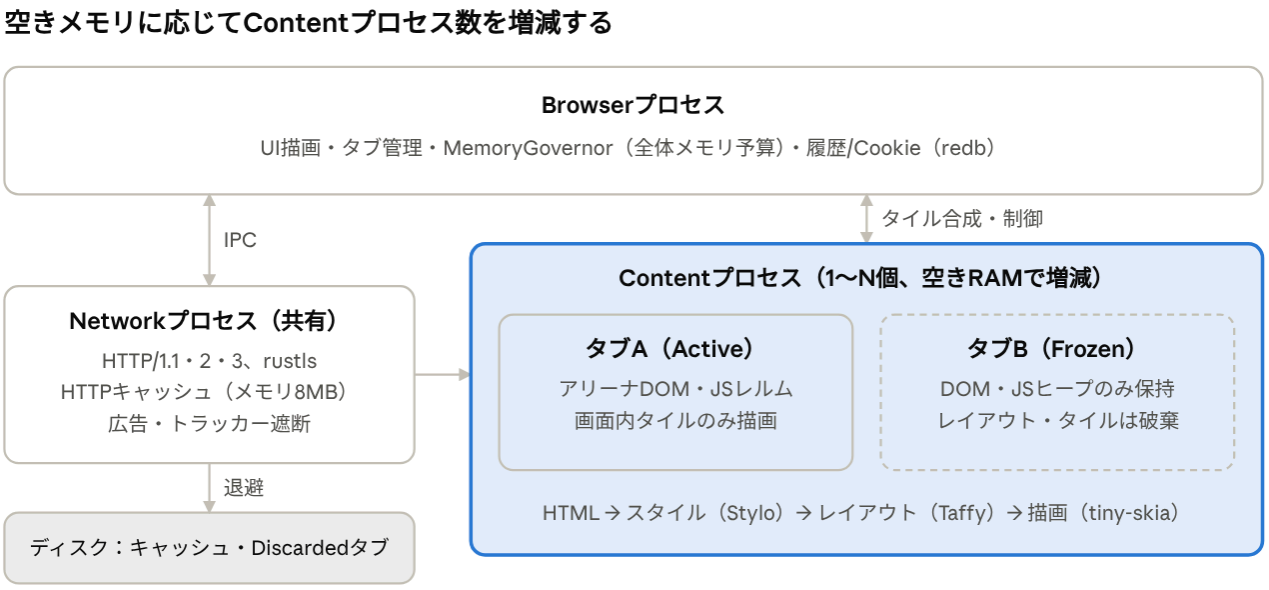
- RAM 2〜4GBの古いPC、Raspberry Pi級のSBC、組み込み端末
- 大量のタブを開いたまま作業する開発者・研究者

非目標（初期版ではやらない）

- 初日から全サイトの完全互換（Google Docs級の重量アプリは段階的に対応）
- WebGL / WebGPU、WebRTC、DRM（EME）
- 拡張機能API（コンテンツブロックは本体に内蔵）

全体アーキテクチャ

プロセス数を空きメモリに応じて増減させる「適応型サイト分離」を採用します。Chromeのような1サイト1プロセスは安全ですが、プロセスごとに数十MBの固定費がかかるためです。



プロセス構成 · 3種類のプロセスとタブの状態

Networkプロセスは全タブで共有し、レスポンスは共有メモリ経由でContentプロセスへ流れます。Frozenタブはレイアウトとタイルを捨て、DOMとJSヒープだけを残します。

プロセス構成

1. **Browserプロセス**：UI、タブ管理、MemoryGovernor（全体のメモリ予算管理）、永続ストレージ。
2. **Networkプロセス**：HTTP/1.1・2・3、TLS、DNS、Cookie、HTTPキャッシュ、コンテンツブロック。全タブで1つを共有。
3. **Contentプロセス（1〜N個）**：HTML/CSS/DOM/JS/レイアウト/ラスタライズ。1プロセス内に複数タブを同居させ、タブごとに独立したアリーナとJSレルムを持つ。

適応ルール

- 空きRAMが十分（例：4GB超）：eTLD+1単位でContentプロセスを分け、最大Nを設定値まで増やす。
- 空きRAMが少ない：Contentプロセスを1〜2個に集約。代わりにSharedArrayBufferと高精度タイマーを無効化し、Spectre系リスクを下げる。
- ログイン済みサイト・決済サイトなど機微なオリジンは、常に専用プロセスに隔離。

データフロー（1ページ表示）

1. NetworkプロセスがレスポンスをIPC（共有メモリのリングバッファ）でストリーム送信。
2. Contentプロセスがバイト列を逐次パースし、DOMをアリーナ上に構築。
3. スタイル計算 → レイアウト → ディスプレイリスト生成。

4. ビューポート＋前後1画面分のタイルだけをラスタライズし、Browserプロセスがウィンドウに合成。

コンポーネント別設計と採用クレート

既存の純Rustクレートを最大限使い、自作は「メモリを左右する部分」（DOMアリーナ、メモリ管理、タブ休止）に集中させます。クレート名は設計時点の候補で、採用前に成熟度とメンテ状況を確認します。

層	採用候補	省メモリ上の設計判断
ウィンドウ・入力	winit	OSネイティブUIツールキットを使わず依存を最小化
ブラウザUI	自エンジンで描画（HTML/CSS製UI）	第2のUIツールキットを載せず、描画系を一本化
HTTP / TLS	hyper、h2、quinn（HTTP/3）、rustls、hickory-dns	受信をストリーム処理し、レスポンス全体をメモリに保持しない
コンテンツブロック	adblock（Brave製の純Rust実装）	広告・トラッカーを読み込まないことが最大の省メモリ策。フィルタはシリアライズ済み形式でmmap
HTMLパーサ	html5ever	トークナイザだけ使い、ツリー構築は自前アリーナDOMへ直接出力
DOM	自作アリーナDOM	後述。ノードは32ビットID、ドキュメント単位で一括解放
CSS・スタイル	Stylo（Servo/Firefoxのスタイルシステム）	スタイル共有キャッシュで同一計算値を共有。互換性を優先
レイアウト	Taffy（block / flex / grid）＋自前のインライン処理	レイアウトツリーは画面外・非アクティブ時に破棄可能な派生データとして扱う
テキスト	Parley、fontique、skrifa / swash	システムフォントをmmapし、全タブでグリフキャッシュを共有
描画	tiny-skia（CPU、既定）、Vello（GPU、任意）	低メモリ機ではCPUタイル描画。GPUテクスチャの共有メモリ消費を避ける
画像	image、zune-jpeg、png、image-webp	表示サイズでデコード、画面外の展開済みビットマップは破棄

層	採用候補	省メモリ上の設計判断
JavaScript	Boa（第一候補）、Nova（調査対象）	JITなしのバイトコードインタプリタ。JITコード領域が不要で、メモリが予測しやすい
永続化	redb	履歴・Cookie・キャッシュ索引を純Rustの組み込みDBに格納
IPC	自作（共有メモリ＋ipc-channel系）	大きなバッファはコピーせず共有メモリで受け渡し

JavaScriptエンジンについて：JITなしは速度で不利ですが、一般的な文書系サイトでは許容範囲です。重いサイトのために、後期フェーズでベースラインJIT（Craneliftを利用）をタブ単位のオプトインで検討します。

省メモリ戦略

効果の大きい順に、タブ休止、データ構造の圧縮、キャッシュ予算の3本柱で削ります。

1. タブのライフサイクル管理

タブは4段階で状態を落とし、段階ごとに再生成できるデータから捨てます。

状態	移行条件（既定値）	保持するもの	捨てるもの	目安
Active	表示中	すべて	なし	制限なし
Background	非表示になった直後	DOM、スタイル、レイアウト、JS	ラスタタイル、画像ビットマップ、GPU資源	約50%減
Frozen	非表示5分、またはメモリ圧迫	DOM、JSヒープ（GC＋コンパクション後）	レイアウトツリー、ディスプレイリスト、スタイル共有キャッシュ	5〜15MB
Discarded	非表示30分、または強い圧迫	ディスク上のスナップショット（URL、スクロール位置、フォーム入力、履歴）	メモリ上の全データ	0.1MB未満

- Frozen中はタイマー・rAF・ネットワークを停止。音声再生中、WebSocket接続中、フォーム入力途中のタブは降格を遅らせる。

- 復帰時は「レイアウトのみ再計算」で済むFrozenを優先し、Discardedは再読み込み＋状態復元。

2. データ構造の圧縮

- **アリーナDOM**：ドキュメントごとに1つのアリーナを持ち、ノードは`u32`のIDで参照。`Rc<RefCell<…>>`の木を避け、参照カウントとポインタの分のメモリを削る。ページ遷移時はアリーナごと解放し、断片化も防ぐ。
- **ノードの小型化**：要素ノードの本体を約32バイトに収め、属性はインライン小配列（2個まで）＋あふれた分だけ別領域。
- **文字列インターン**：タグ名・属性名・クラス名をAtom化（string_cache）。テキストノードは短い文字列をインライン保持（compact_str）。
- **一時領域**：レイアウトやスタイル計算のフレーム内一時データはbumpaloで確保し、フレーム終了時にまとめて捨てる。
- **アロケータ**：mimallocを使い、アイドル時とタブ休止時に未使用ページをOSへ返却（`madvise`）。

3. キャッシュと画像の予算制

- **MemoryGovernor**：Browserプロセスが全体予算を持ち、各サブシステムに上限を配分。OSのメモリ圧迫通知（LinuxのPSI、macOSのmemory pressure、WindowsのQueryMemoryResourceNotification）を受けて、古いタブから順に降格。
- **画像**：圧縮データを保持し、展開は表示サイズで行う（JPEGはDCTスケーリングで縮小デコード）。画面外に出た展開済みビットマップはLRUで破棄。アニメーションGIFは可視時のみデコード。
- **ラスタ**：ビューポートと前後1画面分のタイルのみ保持。1080pで約12MB上限。
- **HTTPキャッシュ**：メモリ上は8MBまで、残りはディスク。
- **JS**：関数本体は初回呼び出しまでバイトコード化しない（遅延コンパイル）。長く呼ばれていない関数のバイトコードは捨て、必要時に再生成。タブごとにヒープ上限を設定。
- **フォント**：システムフォントはmmapで共有し、グリフアトラスは全タブ共通で上限4MB。

セキュリティとサンドボックス

プロセスを減らす分の安全性は、Rustのメモリ安全性とOSサンドボックスで補います。プロセス集約はメモリと安全性のトレードオフなので、既定値はユーザーが選べるようにします。

- **unsafeの封じ込め**：エンジン各crateは`#![forbid(unsafe_code)]`を原則とし、unsafeはFFI・アロケータ・共有メモリIPCの専用crateに限定。そのcrateはMiriとファジングで重点検査。

- **OSサンドボックス**：Contentプロセスはファイル・ネットワーク・プロセス生成を禁止。Linuxはseccomp-bpf＋Landlock、macOSはApp Sandbox、WindowsはAppContainerを使う。
- **集約モードの緩和策**：複数サイト同居時は、SharedArrayBuffer・高精度タイマーを無効化し、オリジンごとにJSレルムとアリーナを分離。
- **TLS**：rustlsのみ。OpenSSL等のC依存を持たない。
- **ファジング**：HTMLパーサ、CSSパーサ、画像デコーダ、IPCメッセージをcargo-fuzzで常時ファジング。

メモリ予算表と計測方法

Wikipedia記事1タブ表示時の予算は合計66MBで、目標70MBに4MBの余裕を残します。数値は設計上の目標値で、試作後に実測で見直します。

サブシステム	予算	超過時の動作
ラスタタイル	12MB	先読みタイルを削減
Browserプロセス本体＋UI	12MB	—
JSヒープ＋バイトコード	10MB	GC実行、未使用バイトコード破棄
DOM＋スタイル	8MB	予算外（ページ依存）として計測のみ
展開済み画像	8MB	画面外ビットマップをLRU破棄
Networkプロセス	6MB	メモリキャッシュをディスクへ退避
フォント・グリフキャッシュ	6MB	グリフアトラスをLRU破棄
レイアウト＋ディスプレイリスト	4MB	画面外サブツリーのレイアウトを破棄

計測方法

- **タグ付きアロケータ**：グローバルアロケータをラップし、サブシステムごとの確保量をカウント。`about:memory`ページで内訳を表示。
- **プロセス全体**：LinuxはPSS（`/proc/<pid>/smaps_rollup`）で計測し、共有ページの二重計上を避ける。
- **ヒーププロファイル**：dhat-rsで確保箇所を特定。
- **CI回帰ゲート**：主要50サイトのスナップショットを読み込み、PSSが前回比5%以上増えたらマージを止める。同条件でFirefox・Chromeと比較した値をダッシュボードに記録。

開発ロードマップとリスク

静的な文書サイトを省メモリで表示できる段階を最初のマイルストーンとし、JSと互換性はその後に広げます。各フェーズは完了条件を満たしてから次へ進みます。

1. **Phase 0：静的ビューア**：HTML/CSS → アリーナDOM → Stylo → Taffy → tiny-skiaで描画。完了条件：JSなしでWikipedia記事を正しく表示し、30MB以下。
2. **Phase 1：ブラウザの骨格**：タブ、履歴、Networkプロセス、rustls、ディスクキャッシュ、コンテンツブロック。完了条件：10タブで250MB以下（JSなし）。
3. **Phase 2：JavaScript**：Boa統合、DOMバインディング、イベント、fetch、タイマー。完了条件：主要50サイト中35サイトが実用可能。
4. **Phase 3：メモリ管理の完成**：MemoryGovernor、Frozen / Discarded、OSメモリ圧迫通知への対応、`about:memory`。完了条件：前述の数値目標をすべて達成。
5. **Phase 4：互換性と速度**：Web Platform Testsの通過率向上、Vello（GPU）描画、ベースラインJITの検討。

主なリスクと対策

リスク	影響	対策
純RustのJSエンジンが遅い	重量級サイトが実用にならない	対象を文書系サイトに絞って開始し、ベースラインJITをオプトインで追加
Web互換性の長いテール	一部サイトが崩れる	WPTを指標に優先度付け。サイト単位の互換性レポート機能を用意
Styloの組み込みとメモリ量	スタイル系が予算超過	早期に実測し、超過ならスタイル共有キャッシュの上限化や自前カスケードを検討
プロセス集約による安全性低下	サイト間攻撃の余地	機微なオリジンの常時隔離、SAB・高精度タイマー無効化、ユーザーが分離度を選択可能に
依存クレートの成熟度	開発停止・仕様変更	重要層は抽象化トレイトの裏に置き、差し替え可能にする